CQRS, Event Sourcing & Caching Cheatsheet

Lessons: 37 — CQRS & Event Sourcing · 38 — Caching Patterns

Examples: 37 · 38

Covers: read/write splitting, event stores, projections, cache strategies, invalidation, stampede protection

Legend: `[*]` = concept or API the lessons have not covered yet

CQRS

```
command                     changes state, returns nothing (or just an ack)
query                       returns state, changes nothing
the split                   different MODELS for writing and reading
write model                 normalized, invariant-enforcing, aggregate-shaped
read model                  denormalized, query-shaped, one table per screen
why                         reads and writes have opposite requirements
the cheap version           same database, different structs and different queries
the full version            separate stores, kept in sync by events
eventual consistency        the read model lags; the UI must tolerate it
read-your-writes            return the new value from the command, or read the
                            write model for that one user
when NOT to                 a CRUD app with one screen — this is pure overhead
```

EVENT SOURCING

```
the idea                    store the EVENTS, not the current state
the state is a fold         state = reduce(apply, events, zero)
event store                 append-only: (stream_id, version, type, payload)
optimistic concurrency      INSERT with expected_version; conflict = someone else won
replay                      rebuild any state, at any past point in time
projections                 build read models by consuming the event stream
snapshots                   every N events, store the folded state so replay is bounded
what you gain               a perfect audit log, temporal queries, new projections
                            built from history you already have
what you pay                no ad-hoc SQL, schema evolution forever, real complexity
NEVER change a stored event   append a correcting event instead
versioning                  upcast old events on read
when to use it              money, compliance, audit, anything where "why" matters
when NOT to                 most CRUD; and never "for the whole system"
```

THE PROJECTION LOOP

```
read events after the last processed position
apply each to the read model
store the new position IN THE SAME transaction as the model update
                            -> at-least-once, and idempotent by position
rebuild                     truncate the read model, replay from 0
lag metric                  head position minus projection position
(a projection is just a consumer with a checkpoint)
```

CACHING: the strategies

```
cache-aside (lazy)           the default. Read: check cache -> miss -> load DB ->
                             fill cache. Write: update DB, then INVALIDATE the key.
read-through                 the cache itself loads on miss (a library does it)
write-through                write cache and DB together; consistent, slower writes
write-behind (write-back)    write cache now, DB later; fast, and you can lose data
refresh-ahead            [*] refresh hot keys before they expire
(cache-aside + short TTL covers 90% of real systems)
```

INVALIDATION

```
TTL                          the only strategy that self-heals; always set one
explicit delete on write     delete the key, don't update it (update races lose)
delete, don't set            two writers setting a key can leave the older value
versioned keys               user:42:v7 — a bump invalidates everything at once
key namespacing              app:entity:id:field, and a global prefix per deploy
tag-based invalidation   [*] group keys so one event clears a set
negative caching             cache "not found" too, with a SHORT TTL
jittered TTLs                ttl + rand(0, 10%) so keys don't all expire together
(there are two hard problems, and this is one and a half of them)
```

STAMPEDE / THUNDERING HERD

```
the problem                  a hot key expires; 1000 requests miss and all hit the DB
singleflight                 collapse concurrent identical loads into ONE call
  var g singleflight.Group
  v, err, shared := g.Do(key, func() (any, error) { return loadFromDB(key) })
  g.DoChan(key, fn)          the channel form, so you can select on ctx
  g.Forget(key)              stop sharing an in-flight result
early recompute              refresh at 80% of the TTL, in the background
locked recompute             SET NX a lock key; one loader, the rest serve stale
serve stale on error         better a 5-minute-old answer than a 500
jitter the TTL               so a whole cohort of keys doesn't expire at once
```

LOCAL vs DISTRIBUTED

```
local (in-process map)       nanoseconds, no network, PER REPLICA
  invalidation is the problem: replica B doesn't know replica A wrote
  fine for: config, feature flags, immutable reference data, short TTLs
distributed (Redis)          shared truth, one network hop, one more thing to run
  fine for: sessions, rate limits, expensive query results, cross-replica state
two-tier                     local (1s TTL) in front of Redis — no hop for hot keys
  invalidate the local tier with a pub/sub message, or just accept 1s of staleness
what NOT to cache            anything you must never serve stale; anything cheap
                             to compute; anything with a 1% hit rate
```

CACHE METRICS & SIZING

```
hit ratio                    below ~80% and the cache may be costing you
latency p99 with and without  the number that justifies it
eviction rate                high = the cache is too small
memory / key count           and a MAX policy (Redis maxmemory + allkeys-lru)
key cardinality              caching per-user data with a 1-hour TTL = a memory bomb
(measure before you cache — most "slow" endpoints are one missing index)
```

TRAPS & MEMORIZE

```
CQRS "everywhere"             two models for a CRUD table is pure cost
event sourcing the whole app  the migration you cannot walk back
mutating a stored event       the one rule of an event store
no snapshots                  a 500k-event stream replays on every load
projection without a checkpoint   a restart replays from zero, or skips events
caching before measuring      you cached the wrong thing
SET instead of DEL on write   the classic lost-update race
no TTL                        one stale key lives until the next deploy
cache key collisions          missing tenant/user in the key = data leak across users
caching the error             a 500 cached for an hour
stampede on a hot key         singleflight exists for exactly this
local cache + N replicas      N different answers to the same question
caching authorization results  a revoked permission still works for an hour
```